Linux setup guide for recording the demo

Smart CMM Quality · Team Xyrus · for the teammate who records the Round 2 video on **Ubuntu** (22.04 or newer). Do the steps in order. Total time: about 30–45 minutes, mostly downloads. Everything the app needs runs in Docker; you do **not** install Python or PostgreSQL yourself.

Step 1: Accept the GitHub invitation

Open the invitation email from GitHub (or https://github.com/notifications) and accept access to `yaswanthme007/smart-cmm-quality`. The repository is private, so you must sign in to GitHub on your laptop in Step 3 before you can clone it.

Step 2: Install the tools (once)

Open a terminal (Ctrl+Alt+T) and run each block.

2a. Basic tools

```
sudo apt update
sudo apt install -y git make curl ca-certificates gh obs-studio
```

No Google Chrome? Install Chromium with `sudo snap install chromium` and use `chromium` instead of `google-chrome` in Step 6.

2b. Docker Engine + Compose (official Docker repository for Ubuntu)

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker $USER
```

Now **log out of Ubuntu and log back in** (or reboot). Without this, Docker says "permission denied".

2c. uv (runs the demo file generator; it downloads Python 3.12 by itself)

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Close the terminal and open a new one.

2d. Check everything

```
docker ps                # must work WITHOUT sudo (an empty table is fine)
docker compose version
make --version
uv --version
gh --version
```

If `docker ps` says permission denied, you have not logged out and back in after 2b.

Step 3: Sign in to GitHub and get the code

```
gh auth login            # choose GitHub.com, HTTPS, "Login with a web browser"
cd ~/Desktop
gh repo clone yaswanthme007/smart-cmm-quality
cd smart-cmm-quality
git checkout dev         # all the work is on the dev branch
```

Step 4: Create your settings file (.env)

Secrets are never stored in Git, so every laptop makes its own `.env`.

```
cp .env.example .env
openssl rand -hex 32     # copy the long output for JWT_SECRET
nano .env                # edit, then Ctrl+O, Enter, Ctrl+X to save and exit
```

Change these lines (example values; choose your own):

Line in .env	Set it to
`POSTGRES_PASSWORD=`	any password, e.g. `demo-db-2026`
`DATABASE_URL=`	same password inside: `postgresql+psycopg://cmm:demo-db-2026@localhost:5432/cmm_quality`
`JWT_SECRET=`	the long value from `openssl rand -hex 32`
`DEMO_USER_PASSWORD=`	anything (one-click demo login means you never type it)
`DEMO_MODE=`	keep `true`
`ESCALATION_PROFILE=`	keep `demo` (escalation after 60 s and 120 s)
`AI_ENABLED=`	keep `false` (only needed if you show PDF extraction with a Groq key)

Step 5: Start the system and load the demo data

```
make up            # first time builds everything: 5–15 minutes depending on internet
make demo-reset    # about 20 seconds: demo parts, 30 days of synthetic history, tool-wear drift
```

Check in the browser:

Open	You should see
http://localhost:8000/api/health	`"status":"ok"` (worker shows `ok` within about 10 seconds)
http://localhost:5173	the login page with one-click demo-user buttons
http://localhost:8025	Mailpit, an empty demo inbox

Step 6: Open one browser window per role

Each role needs its **own browser profile**; windows of the same profile share the login and would all become the same user. With Google Chrome (or `chromium` instead of `google-chrome`), run:

```
google-chrome --user-data-dir=$HOME/.demo-qhead      "http://localhost:5173/login?as=qhead" &
google-chrome --user-data-dir=$HOME/.demo-owner      "http://localhost:5173/login?as=owner.vmc02" &
google-chrome --user-data-dir=$HOME/.demo-supervisor "http://localhost:5173/login?as=supervisor" &
google-chrome --user-data-dir=$HOME/.demo-mail       "http://localhost:8025" &
```

Each window signs in automatically (no password). In every window: zoom to 110% (Ctrl and +), hide the bookmarks bar (Ctrl+Shift+B). In the owner window, click once anywhere so the browser allows the alert sound.

Step 7: Set up OBS Studio for recording

1. Open OBS → Settings → Video: Base and Output resolution **1920×1080**, 30 FPS. 2. Settings → Output: Recording format **MP4** (or MKV and remux to MP4 afterwards). 3. In Sources click + → **Screen Capture (PipeWire)** (Ubuntu uses Wayland; choose your screen when asked). On an X11 session the source is called **Screen Capture (XSHM)**. 4. Add **Audio Input Capture** for your microphone; right-click it → Filters → add Noise Suppression. 5. Turn on Do Not Disturb (Ubuntu notification menu) and close chat apps.

Step 8: Dry run, then reset

```
make demo-drop     # the live step: a Not OK CMM file lands in the watched folder
```

The owner window should show a red alert within about a second, and Mailpit should show the email. After about 60 seconds the supervisor window gets the escalation. Then reset for the real take:

```
make demo-reset    # always run this before each take; the live NC will be NC-2026-00042
```

Reload the windows (F5); they stay signed in.

Step 9: Record

Follow **docs/DEMO_SCRIPT.pdf** (Part 5: shot list and narration). Record each part as a separate clip, run `make demo-reset` between takes, and never edit numbers afterwards.

Daily commands

Command	What it does
`make up`	start everything (also after `git pull`, it rebuilds what changed)
`make demo-reset`	back to the exact starting state of the demo (≈20 s)
`make demo-drop`	live step: drop the Not OK file
`make logs`	see what the services are doing (Ctrl+C to stop watching)
`make down`	stop everything
`git pull`	get the latest changes from the team, then `make up`

Troubleshooting

Problem	Fix
`permission denied ... /var/run/docker.sock`	`sudo usermod -aG docker $USER`, then log out and log back in
`port is already allocated` for 5432	you have PostgreSQL installed: `sudo systemctl stop postgresql`, or set `DB_PORT=5433` in `.env` and run `make up`
Another port busy (8000, 5173, 1025, 8025)	close the program using it (`sudo ss -ltnp | grep 8000` shows which)
`gh repo clone` says not found	the invitation is not accepted yet, or `gh auth login` used a different GitHub account
Web page shows a red Vite error after `git pull`	run `make up` again (rebuilds with new dependencies)
Login page has no demo-user buttons	check `DEMO_MODE=true` in `.env`, then `make up`
No alert pops up in the owner window	each role must have its own `--user-data-dir`; the dot next to "Demo data" must be green
`make demo-drop` fails with uv not found	open a new terminal after installing uv (Step 2c)
OBS records a black screen	use the PipeWire source on Wayland, or log in with "Ubuntu on Xorg" and use XSHM

Problem	Fix
Anything strange before a take	`make demo-reset` and reload the windows

All data shown is synthetic demo data. Say "Demo data" if anyone asks.